

PARALLEL VISION

*Plateforme de traitement parallèle d'images
avec benchmarking intégré*

FastAPI

React

OpenCV

Docker

NumPy



SOMMAIRE

01 Vue d'ensemble & Objectifs

02 Architecture Frontend React

03 Architecture Backend Python

04 Architecture Parallèle & IPC

05 Synchronisation (Barbier & Philo)

06 Benchmarking & Résultats

07 API REST & Docker

VUE D'ENSEMBLE

Objectif & Périmètre du projet

Objectif

L'utilisateur charge un grand nombre d'images depuis une interface web React. Le backend Python les traite en parallèle (multiprocessing + threading) via FastAPI et retourne les résultats enrichis : contours, histogramme, détection de visages.

5

configs
benchmark

3

mécanismes
IPC

2

problèmes
synchro

∞

images
scalable

Traitements par image :

- Redimensionnement
- Niveaux de gris
- Détection contours (Canny)
- Histogramme
- Détection visages (Haar Cascade)
- Image annotée

FRONTEND REACT

Interface moderne — Material UI



UploadPanel

Upload multiple images
Configuration processus / threads
Démarrage du traitement



ProgressPanel

Barre de progression live
Nombre d'images traitées
État du job (running / done)



ResultsPanel

Galerie : gris · contours
Image annotée (visages)
Temps par image



FacePanel

Bar chart face counts
Galerie images avec visages
Bounding boxes par image



BenchmarkPanel

Tableau speedup / efficacité
Comparaison configs P×T
Export CSV



Polling toutes les secondes via /status — mise à jour live de la progression et des résultats.

BACKEND PYTHON

FastAPI · Multiprocessing · Threading · OpenCV

app/main.py

API REST endpoints
/upload /start /status
/results /benchmark

app/controller.py

MasterController
Gestion workers
Collector thread

app/processing.py

process_image()
Thread face detection
CV pipeline parallèle

app/config.py

AppConfig frozen
Processes · Threads
Tokens · Bins

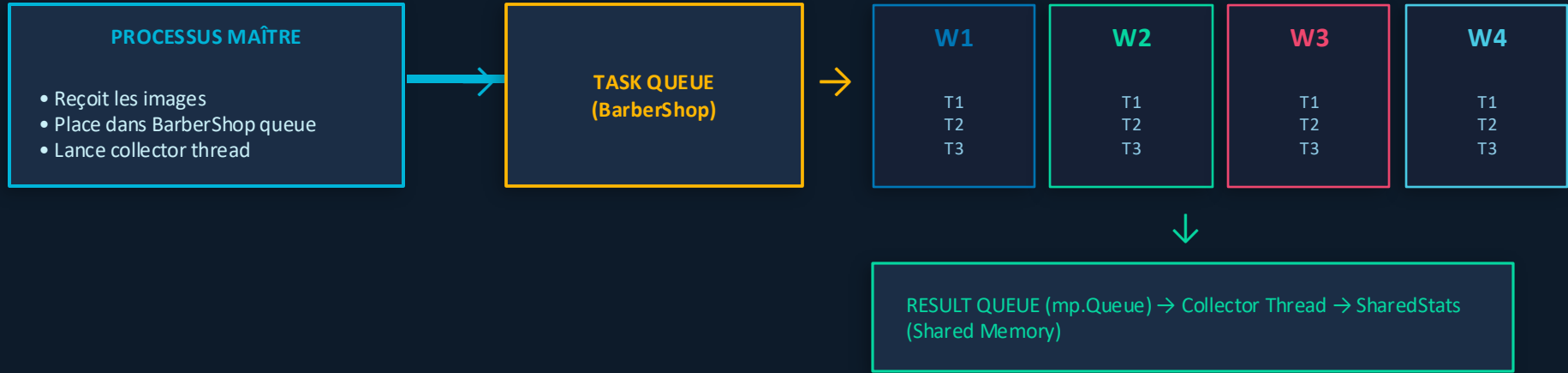
workers/worker.py

WorkerProcess
Thread pool interne
Acquisition tokens

app/models.py

Pydantic schemas
ResultItem · BenchmarkRow
StatusResponse

ARCHITECTURE PARALLÈLE



MÉCANISMES IPC

Pipes

Commandes Maître → Workers (STOP / signal)

Shared Memory

Statistiques globales (SharedStats / numpy)

Message Queue

Résultats Workers → Collector Thread

SYNCHRONISATION

A · Problème du Barbier Endormi

Images	= Clients
Workers	= Barbiers
Task Queue	= Salle d'attente

Primitives utilisées

- Semaphore — contrôle d'accès à la queue
- Lock — exclusion mutuelle sur le compteur
- Queue — file FIFO multiprocessus
- count Value — nombre de tâches en attente

Les workers dorment (semaphore.acquire) lorsqu'aucune image n'est disponible, garantissant zéro busy-wait.

B · Problème des Philosophes Dîneurs

CPU Token	= Semaphore (cpu_tokens)
GPU Token sim.	= Semaphore (gpu_tokens)
Workers	= Philosophes

Prévention du deadlock (TokenPair)

- acquire_cpu (timeout 0.2s)
- Si échec → retry sans bloquer
- acquire_gpu (timeout 0.2s)
- Si échec → release_cpu + sleep(0.01)
- Traitement dès les 2 tokens acquis
- release_gpu → release_cpu après fin

BENCHMARKING

Mesure du speedup et de l'efficacité

Config	Processus	Threads	Unités parallèles	Speedup attendu
C1	1	1	1	Baseline
C2	1	4	4	~3.5×
C3	2	4	8	~6×
C4	4	4	16	~10×
C5	8	8	64	~15×

Temps total

`perf_counter() end - start`

Temps moyen

`total_time / nb_images`

CPU %

`psutil.cpu_percent()`

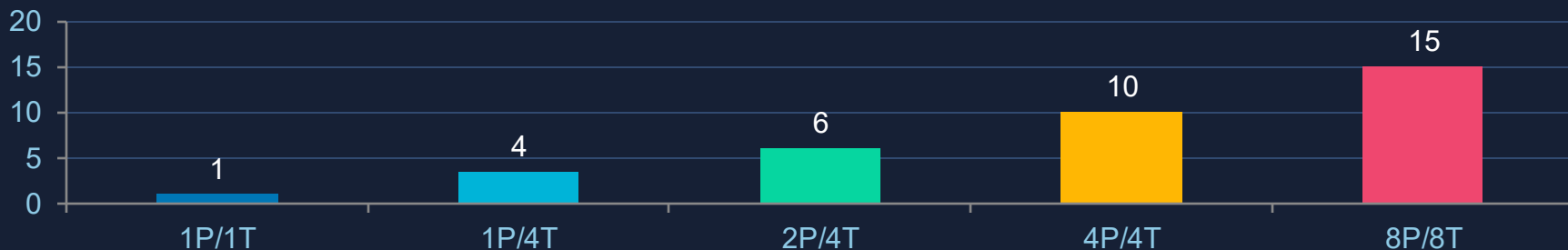
Speedup

$T_{baseline} / T_{config}$

Export CSV

`write_csv()` → `benchmark/results.csv`

Speedup estimé par configuration



API REST & DOCKER

POST

/upload

Upload images (multipart/form-data)

POST

/start

Lance le traitement (processes, threads)

GET

/status

État du job + statistiques en temps réel

GET

/results

Résultats + images base64 encodées

GET

/benchmark

Lance ou récupère le benchmark CSV

DÉPLOIEMENT DOCKER

[backend/Dockerfile](#)

Python 3.11 · FastAPI · OpenCV · uvicorn

[frontend/Dockerfile](#)

Node 20 · Vite · React · nginx

[docker-compose.yml](#)

Services backend + frontend + réseau partagé

CONCLUSION



Architecture parallèle complète : multiprocessing + threading + IPC (Pipes, Shared Memory, Queue)



Synchronisation avancée : Barbier Endormi (zéro busy-wait) + Philosophes Dîneurs (anti-deadlock)



Détection de visages parallélisée : thread OpenCV Haar Cascade concurrent au pipeline CV



Benchmarking complet : 5 configs · speedup · efficacité · export CSV · affichage React



Stack moderne : FastAPI · React/MUI · Docker · Pydantic · psutil · NumPy · OpenCV